

## TECNOLOGÍAS DE LA INFORMACIÓN

### Desarrollo de Sistemas Informáticos

4 créditos



#### Profesores:

Ing. Jorge Arun Zambrano Cedeño Mg.

| Titulaciones                                    | Semestre      |
|-------------------------------------------------|---------------|
| • <i>Tecnologías de la Información en Línea</i> | <i>Quinto</i> |

NOMBRES Y APELLIDOS: ALBA ALEXANDRA SILVA PILA

PARALELO: A

SEMESTRE: QUINTO SEMESTRE

ACTIVIDAD: ACTIVIDAD 1 ESTUDIO DE CASO – SISTEMA DE GESTIÓN DE INCIDENCIAS (HELP DESK)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



## Actividades a desarrollar en la Unidad 1

### Actividad # 1

#### Actividad 1: Estudio de Caso – Sistema de Gestión de Incidencias

##### 1. Introducción

El presente estudio de caso tiene como objetivo diseñar la arquitectura lógica de un sistema Help Desk orientado a la gestión de incidencias técnicas dentro de un entorno de infraestructura tecnológica.

El desarrollo del proyecto permite aplicar conceptos fundamentales de ingeniería de software, diagramas UML, arquitectura MVC y patrones de diseño, con la finalidad de construir una solución organizada, mantenible y escalable.

Además, se implementan buenas prácticas de control de versiones mediante Git y Gitflow, fortaleciendo la trazabilidad y organización del proceso de desarrollo.

##### 2. Descripción de la Actividad

El sistema Help Desk tiene como finalidad gestionar de manera eficiente los incidentes reportados dentro de la infraestructura tecnológica de un Data Center, permitiendo registrar, clasificar, asignar y dar seguimiento a los problemas reportados por los usuarios.

La implementación de este sistema permitirá optimizar los tiempos de respuesta, mejorar la organización de incidencias y mantener un control adecuado sobre el estado de cada ticket generado. Además, contribuirá al cumplimiento de los acuerdos de nivel de servicio (SLA), garantizando una atención técnica más eficiente y estructurada.



## 2.1. Identificación de Actores Principales

### 1. Usuario Final - Personal Operativo:

Es el encargado de reportar incidentes relacionados con fallos en equipos, redes o sistemas informáticos. Su participación es fundamental para iniciar el flujo de atención del ticket.

### 2. Técnico de Soporte

Analiza los incidentes registrados, diagnostica las causas del problema y ejecuta las acciones necesarias para su solución, manteniendo actualizado el estado del ticket.

### 3. Administrador de Sistemas

Supervisa el funcionamiento global de la plataforma Help Desk, controla los tiempos de atención y administra la asignación de recursos técnicos.

## 2.2.Requerimientos funcionales

| ID    | Requerimiento       | Descripción                                                                            |
|-------|---------------------|----------------------------------------------------------------------------------------|
| RF-01 | Creación de Tickets | Permite al usuario registrar un fallo describiendo el problema, ubicación y severidad. |
| RF-02 | Categorización      | Clasifica automáticamente el incidente según el área: Red, Hardware o Software.        |
| RF-03 | Asignación Dinámica | Asigna el ticket al técnico especializado disponible en el área correspondiente.       |



|       |                    |                                                                                            |
|-------|--------------------|--------------------------------------------------------------------------------------------|
| RF-04 | Gestión de Estados | Flujo de resolución:<br>Registrado -> Asignado -><br>En Proceso -> Resuelto -><br>Cerrado. |
|-------|--------------------|--------------------------------------------------------------------------------------------|

Los requerimientos funcionales representan las acciones principales que el sistema debe ejecutar para garantizar un correcto funcionamiento del proceso de gestión de incidencias. Estos permiten automatizar tareas y mejorar la administración de tickets dentro de la organización.

### 2.3. Requerimientos No Funcionales

#### RNF-01 (Escalabilidad)

El sistema deberá soportar múltiples usuarios conectados simultáneamente y un crecimiento progresivo en el número de incidencias registradas.

#### RNF-02 (Mantenibilidad)

El desarrollo del software seguirá buenas prácticas de programación y principios SOLID, facilitando futuras actualizaciones y correcciones.

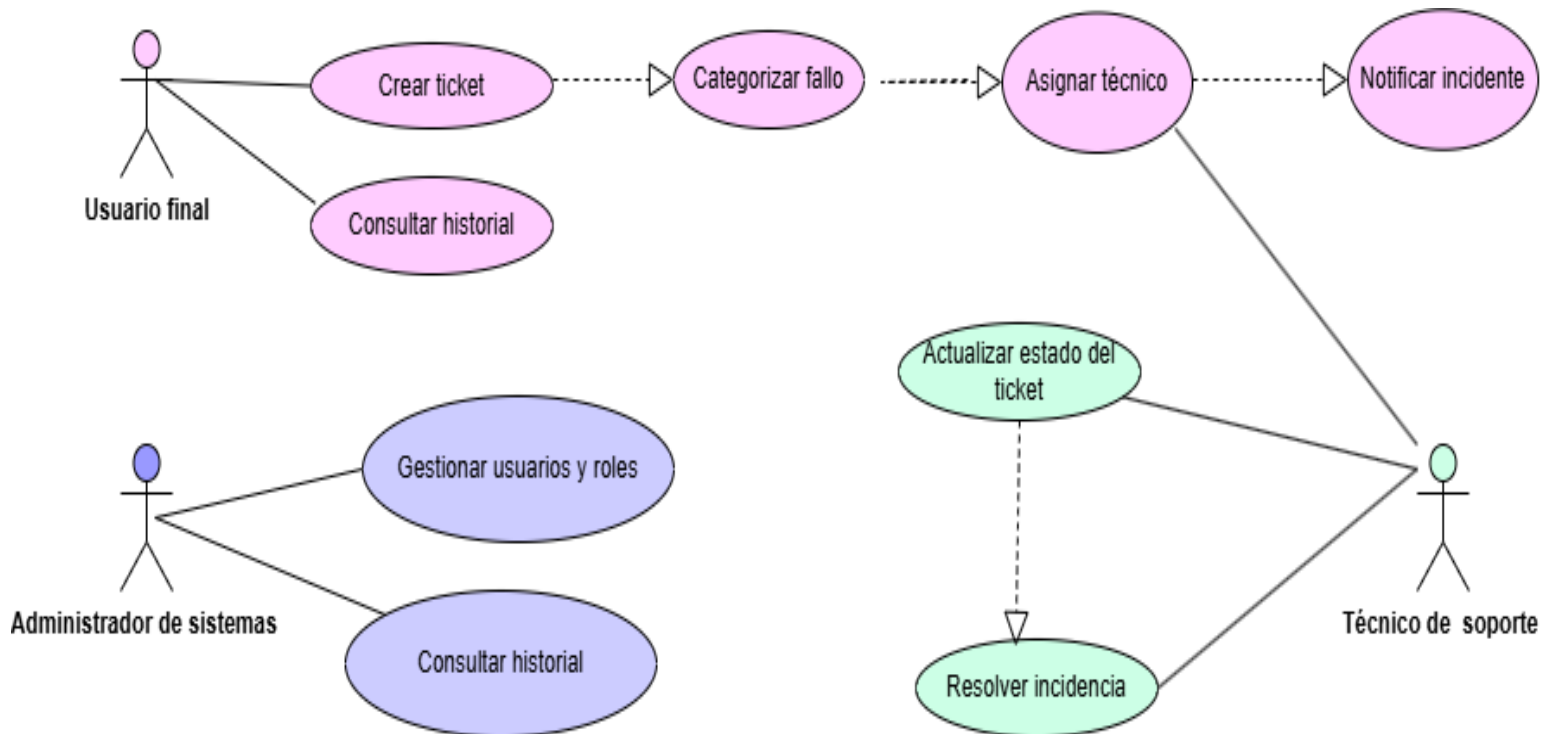
#### RNF-03 (Trazabilidad)

Todas las modificaciones realizadas al sistema deberán mantenerse registradas mediante herramientas de control de versiones como Git y Gitflow.

### 3. Diagrama de Casos de Uso

El siguiente diagrama representa la interacción entre los actores del sistema y las funcionalidades principales del Help Desk, permitiendo visualizar cómo se gestionan los incidentes.

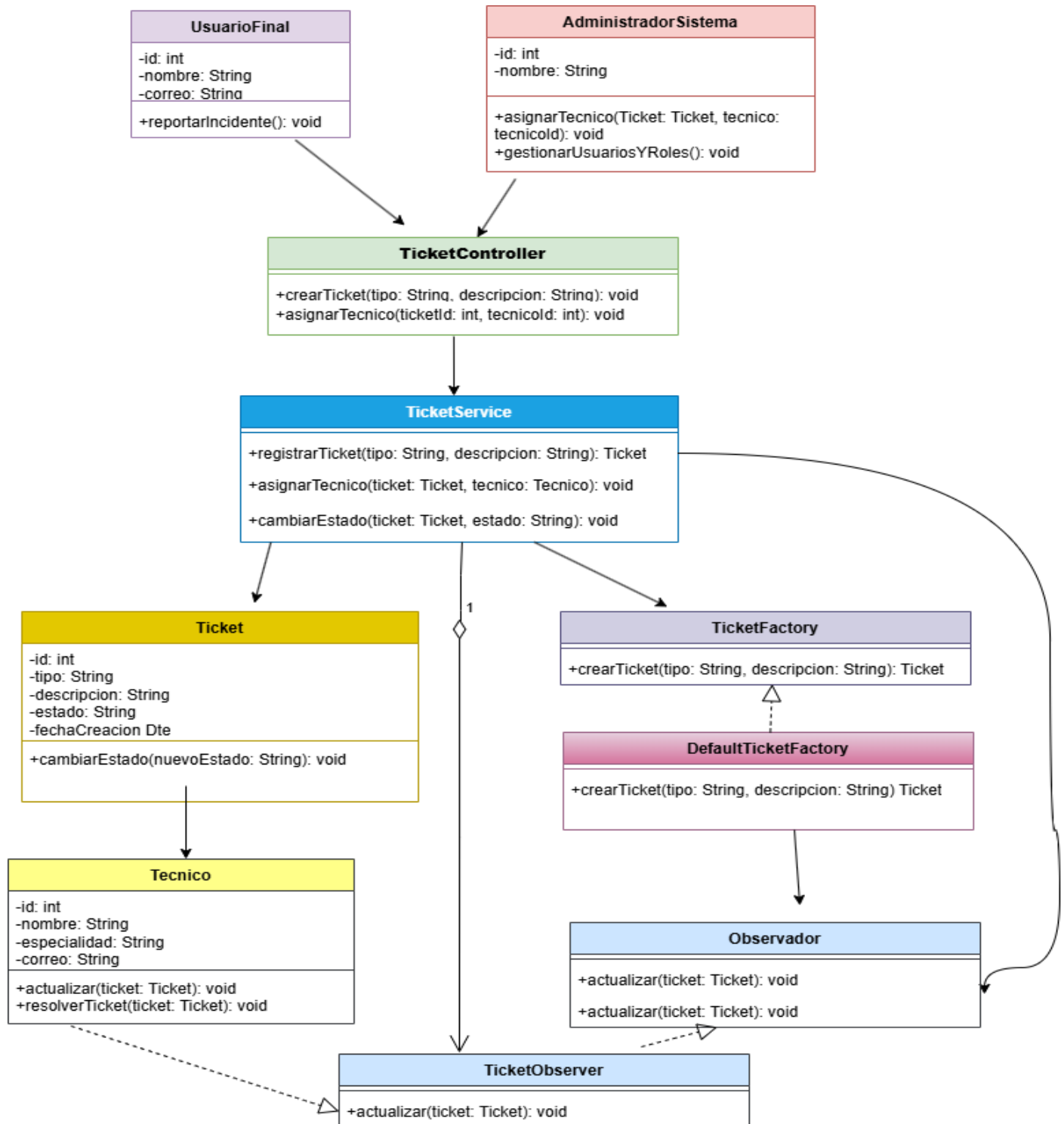
**Figura 1. Diagrama de Casos de Uso.**



### 3.1. Diagrama de Clases UML (Principios SOLID)

El siguiente diagrama muestra la estructura del sistema, incluyendo clases, atributos y relaciones, aplicando principios SOLID para garantizar un diseño mantenible y escalable.

**Figura 2. Diagrama de Clases UML.**





### **3.2. Aplicación de Principios SOLID**

En el diseño del sistema Help Desk se aplican principios SOLID con el objetivo de mejorar la mantenibilidad, escalabilidad y reutilización del software:

#### **3.3. Principio de Responsabilidad Única (SRP)**

Cada clase tiene una única responsabilidad. Por ejemplo, la clase TicketService se encarga exclusivamente de la lógica de negocio, mientras que TicketController gestiona las solicitudes del usuario.

#### **3.4. Principio de Abierto/Cerrado (OCP)**

El sistema está diseñado para permitir la extensión sin modificar el código existente. Esto se evidencia mediante el uso del patrón Factory Method, que permite agregar nuevos tipos de incidentes sin alterar las clases existentes.

#### **3.5. Principio de Sustitución de Liskov (LSP)**

Las clases derivadas pueden sustituir a sus clases base sin alterar el comportamiento del sistema, especialmente en la creación de distintos tipos de tickets.

#### **3.6. Principio de Segregación de Interfaces (ISP)**

Se evita el uso de interfaces grandes y se implementan interfaces específicas para cada funcionalidad, como en el uso del Observador.

#### **3.7. Principio de Inversión de Dependencias (DIP)**

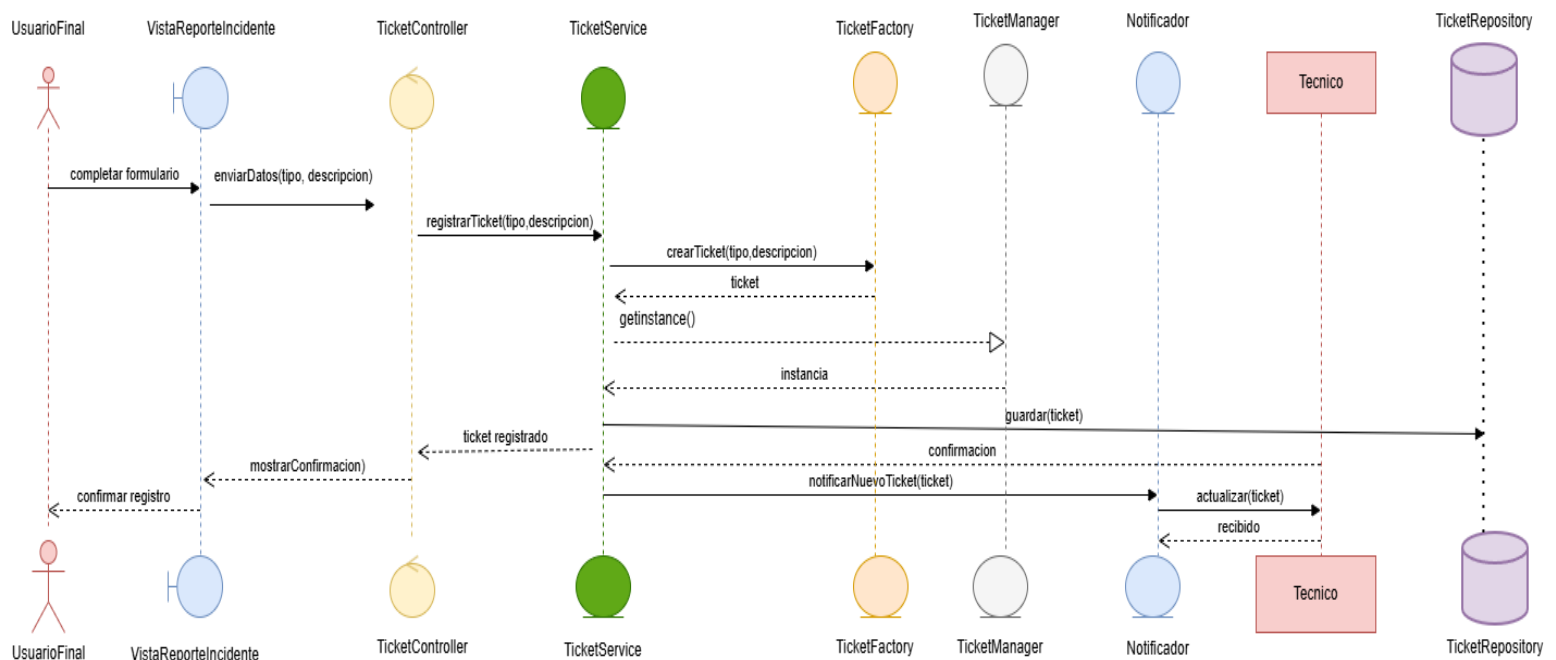
Las clases de alto nivel no dependen de implementaciones concretas, sino de abstracciones, lo que se observa en la relación entre el sistema de notificación y los observadores.

### 3.8. Diagrama de Secuencia

El diagrama de secuencia muestra el proceso completo de gestión de un incidente, desde que el usuario registra un ticket hasta que el técnico lo resuelve.

Se evidencia la interacción entre el controlador, el servicio de tickets y el sistema de notificaciones, aplicando el patrón Observer para informar a los técnicos en tiempo real.

**Figura 3. Diagrama de Secuencia.**



### 2.4. Arquitectura General (MVC)

La arquitectura Modelo Vista Controlador (MVC) permite separar la lógica del negocio, la interfaz gráfica y el control de procesos, facilitando la organización del sistema y mejorando la mantenibilidad del código.

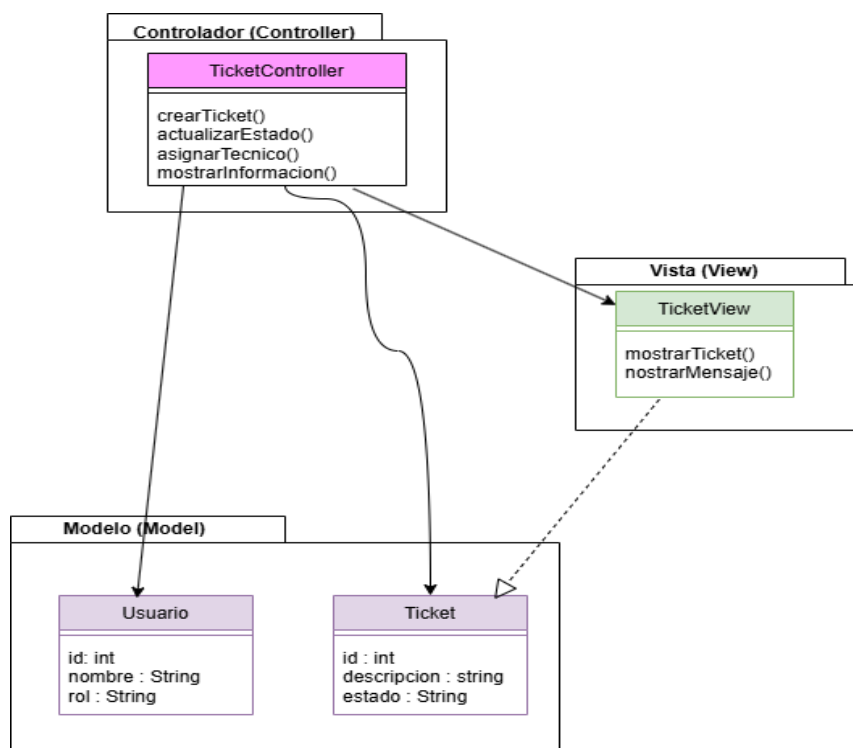


Esta arquitectura favorece el trabajo colaborativo entre desarrolladores, permitiendo realizar modificaciones en una capa sin afectar directamente las demás.

Se ha seleccionado el patrón arquitectónico Modelo Vista Controlador (MVC) para estructurar la solución:

- **Modelo:** Contiene las entidades (Ticket, Usuario) y la lógica de persistencia.
- **Vista:** Interfaces para el reporte de incidentes y el dashboard técnico.
- **Controlador:** Orquesta el flujo de datos, procesando las solicitudes de la vista.

**Figura 4. Arquitectura MVC.**



## 4. Aplicación de patrones de diseño

### 4.1. Singleton (Gestión de Conexión)



### **Problema**

Múltiples conexiones simultáneas a la base de datos pueden generar sobrecarga y consumo innecesario de recursos.

### **Solución**

El patrón Singleton garantiza que exista una única instancia de conexión durante la ejecución del sistema.

### **Justificación técnica**

Permite optimizar recursos, evitar inconsistencias y asegurar un acceso controlado a la base de datos.

## **4.2. Factory Method (Creación de Incidentes)**

### **Problema**

La creación de diferentes tipos de incidentes (Hardware, Red, Software) puede generar dependencia directa de clases concretas.

### **Solución**

El patrón Factory Method delega la creación de objetos a subclases especializadas.

### **Justificación técnica**



Facilita la extensibilidad del sistema permitiendo agregar nuevos tipos de incidentes (por ejemplo, Cloud) sin modificar el código existente, cumpliendo el principio Open/Closed.

#### **4.3. Observer (Notificaciones Automáticas)**

##### **Problema**

La necesidad de notificar a múltiples técnicos cuando ocurre un evento (nuevo ticket o cambio de estado).

##### **Solución**

El patrón Observer permite que múltiples objetos observen y reaccionen automáticamente a cambios en el sistema.

##### **Justificación técnica**

Mejora la comunicación en tiempo real, implementa programación reactiva y desacopla el sistema de notificaciones.

#### **5. Conclusión**

El desarrollo del sistema Help Desk permitió aplicar de manera práctica conceptos de análisis y diseño de software, así como el uso de diagramas UML, arquitectura MVC y patrones de diseño.



Además, se evidenció la importancia de aplicar principios SOLID y metodologías de control de versiones como Gitflow, lo cual facilita la construcción de sistemas escalables, mantenibles y organizados.

Finalmente, este proyecto fortaleció la capacidad de diseñar soluciones tecnológicas alineadas a buenas prácticas profesionales, orientadas a la gestión eficiente de incidencias en entornos de infraestructura tecnológica.

## 6. Evidencias

### 6.1. Uso de Gitflow en el desarrollo del sistema

Para el desarrollo del sistema Help Desk se implementó el modelo de trabajo **Gitflow**, el cual permite organizar el control de versiones de forma estructurada y profesional.

### 6.2. Creación de ramas

#### 1. Rama main

Se utilizó para almacenar versiones estables del sistema listas para producción.

El proyecto helpdesk-data-center fue inicializado en Git y configurado para trabajar sobre la rama **main**, la cual se estableció como rama principal mediante el comando `git branch -M main`, posteriormente, se vinculó al repositorio remoto en GitHub con `git remote add origin` y se sincronizó con `git push -u origin main`, quedando la rama local asociada directamente a `origin/main`, en esta rama se registraron los primeros cambios, incluyendo la creación del archivo `README.md`.



```
MINGW64/c/HelpDesk
hp@ALEXITA MINGW64 ~ (main)
$ pwd
/c/Users/hp
hp@ALEXITA MINGW64 ~ (main)
$ cd ..
hp@ALEXITA MINGW64 /c/Users
$ cd ..
hp@ALEXITA MINGW64 /c
$ mkdir HelpDesk
hp@ALEXITA MINGW64 /c
$ git --version
git version 2.54.0.windows.1
hp@ALEXITA MINGW64 /c
$ git config --global user.name "Alexandra Silva"
error: invalid key: user.nameAlexandra Silva
hp@ALEXITA MINGW64 /c
$ ^[[200~git config --global user.name "Alexandra Silva"
bash: $'\E[200~git': command not found
hp@ALEXITA MINGW64 /c
$ git config --global user.name "Alexandra Silva"
hp@ALEXITA MINGW64 /c
$ git confi --global user.name "asilva6635@utm.edu.ec"
git: 'confi' is not a git command. See 'git --help'.
The most similar command is
  config
hp@ALEXITA MINGW64 /c
$ git config --globa user.name "asilva6635@utm.edu.ec"
error: key does not contain a section: user.name
hp@ALEXITA MINGW64 /c
$ git config --global user.email "asilva6635@utm.edu.ec"
hp@ALEXITA MINGW64 /c
$ ld
bash: ld: command not found
hp@ALEXITA MINGW64 /c
$ ls
$Recycle.Bin/      HelpDesk/      System.sav@
$WinREAgent/      OneDriveTemp/  Users/
$Windows.~WS/      PerfLogs/      Windows/
```

```
MINGW64/c/Users/hp/helpdesk-data-center
$ cd helpdesk-data-center
hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git init
Initialized empty Git repository in C:/Users/hp/helpdesk-data-center/.git/
hp@ALEXITA MINGW64 ~/helpdesk-data-center (master)
$ echo "# helpdesk-data-center" >> README.md
hp@ALEXITA MINGW64 ~/helpdesk-data-center (master)
$ ls
README.md
hp@ALEXITA MINGW64 ~/helpdesk-data-center (master)
$ git add .
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
hp@ALEXITA MINGW64 ~/helpdesk-data-center (master)
$ git commit -m "primer commit del proyecto"
[master (root-commit) e17640c] primer commit del proyecto
1 file changed, 1 insertion(+)
 create mode 100644 README.md
hp@ALEXITA MINGW64 ~/helpdesk-data-center (master)
$ git branch -M main
hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git remote add origin https://github.com/asilva6635-glitch/helpdesk-data-center.git
hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 243 bytes | 243.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/asilva6635-glitch/helpdesk-data-center.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ mkdir diagrams
hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git add .
hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git commit -m "Agregada carpeta diagrams con UML"
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git push
Everything up-to-date
hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$
```



## 2. Rama master

Aunque la rama principal del proyecto se configuró inicialmente como main, se creó la rama master para organizar y centralizar todos los recursos del repositorio, en esta rama se almacenan:

- La carpeta Diagramas, que contiene los archivos UML (Casos de Uso, Clases, Secuencia, Arquitectura MVC).
- La carpeta destinada al informe en PDF, con el objetivo de mantener la documentación junto al desarrollo.

De esta forma, la rama master funciona como un contenedor completo del proyecto, integrando tanto los artefactos técnicos.

```
MINGW64:/c/HelpDesk
nothing to commit, working tree clean

hp@ALEXITA MINGW64 /c/HelpDesk (master)
$ git branch
* master

hp@ALEXITA MINGW64 /c/HelpDesk (master)
$ git push -u origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 6.93 KiB | 3.47 MiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/asilva6635-glitch/helpdesk-data-center/pull/new/master
remote:
To https://github.com/asilva6635-glitch/helpdesk-data-center.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

hp@ALEXITA MINGW64 /c/HelpDesk (master)
$
```



### 3. Rama develop

Permitió integrar los avances de desarrollo de manera progresiva.

```
MINGW64:/c/Users/hp/helpdesk-data-center
$ cd helpdesk-data-center

hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ pwd
/c/Users/hp/helpdesk-data-center

hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git branch
* main

hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git branch develop

hp@ALEXITA MINGW64 ~/helpdesk-data-center (main)
$ git checkout develop
Switched to branch 'develop'

hp@ALEXITA MINGW64 ~/helpdesk-data-center (develop)
$ git add .

hp@ALEXITA MINGW64 ~/helpdesk-data-center (develop)
$ git commit -m "primer commit en develop"
On branch develop
nothing to commit, working tree clean
```

### 4. Rama feature/casos-uso

Se creó la rama feature/casos-uso a partir de develop mediante el comando `git checkout -b feature/casos-uso`, en esta rama se añadieron los diagramas de casos de uso y se realizó el push al repositorio remoto con `git push -u origin feature/casos-uso` y el sistema confirmó la vinculación de la rama local con `origin/feature/casos-uso` y generó el enlace para la creación del pull request en GitHub, evidenciando que el trabajo quedó correctamente registrado en el repositorio.



```
MINGW64/c/Users/hp/helpdesk-data-center

hp@ALEXITA MINGW64 ~ (develop)
$ cd helpdesk-data-center

hp@ALEXITA MINGW64 ~/helpdesk-data-center (develop)
$ git branch
* develop
main

hp@ALEXITA MINGW64 ~/helpdesk-data-center (develop)
$ git checkout -b feature/casos-uso
Switched to a new branch 'feature/casos-uso'

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/casos-uso)
$ git branch
* feature/casos-uso
develop
main

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/casos-uso)
$ git add .

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/casos-uso)
$ git commit -m "Agregado diagrama de casos de uso"
On branch feature/casos-uso
nothing to commit, working tree clean

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/casos-uso)
$ git push -u origin feature/casos-uso
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/casos-uso' on GitHub by visiting:
remote:   https://github.com/asilva6635-glitch/helpdesk-data-center/pull/new/feature/casos-uso
remote:
to https://github.com/asilva6635-glitch/helpdesk-data-center.git
* [new branch]      feature/casos-uso -> feature/casos-uso
branch 'feature/casos-uso' set up to track 'origin/feature/casos-uso'.

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/casos-uso)
```

## 5. Rama feature/casos-uso

Se creó la rama feature/clases-uml a partir de develop mediante el comando `git checkout -b feature/clases-uml`, posteriormente, se verificó la existencia de las ramas en el repositorio local y se realizó el push al repositorio remoto con `git push -u origin feature/clases-uml`, para lo cual sistema confirmó la vinculación de la rama local con `origin/feature/clases-uml` y generó el enlace para la creación del pull request en GitHub, evidenciando que el trabajo quedó correctamente registrado en el repositorio.





```
MINGW64:/c:/Users/hp/helpdesk-data-center
$ git checkout -b feature/clases-uml
Switched to a new branch 'feature/clases-uml'

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/clases-uml)
$ git branch
  develop
  feature/casos-uso
* feature/clases-uml
  main

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/clases-uml)
$ git push -u origin feature/clases-uml
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/clases-uml' on GitHub by visiting:
remote:   https://github.com/asilva6635-glitch/helpdesk-data-center/pull/new/
feature/clases-uml
remote:
To https://github.com/asilva6635-glitch/helpdesk-data-center.git
 * [new branch]      feature/clases-uml -> feature/clases-uml
branch 'feature/clases-uml' set up to track 'origin/feature/clases-uml'.

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/clases-uml)
```

## 6. Rama *feature/sequencia*

Se creó la rama *feature/sequencia* a partir de *feature/clases-uml* mediante el comando `git checkout -b feature/sequencia`. Posteriormente, se añadieron los archivos correspondientes y se realizó el push al repositorio remoto con `git push -u origin feature/sequencia`, el sistema confirmó la vinculación de la rama local con `origin/feature/sequencia` y generó el enlace para la creación del pull request en GitHub.

```
MINGW64:/c:/Users/hp/helpdesk-data-center

hp@ALEXITA MINGW64 ~ (feature/clases-uml)
$ cd helpdesk-data-center

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/clases-uml)
$ git branch
  develop
  feature/casos-uso
* feature/clases-uml
  main

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/clases-uml)
$ git checkout -b feature/sequencia
Switched to a new branch 'feature/sequencia'

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/sequencia)
$ git add .

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/sequencia)
$ git push -u origin feature/sequencia
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/sequencia' on GitHub by visiting:
remote:   https://github.com/asilva6635-glitch/helpdesk-data-center/pull/new/feature/sequencia
remote:
To https://github.com/asilva6635-glitch/helpdesk-data-center.git
 * [new branch]      feature/sequencia -> feature/sequencia
branch 'feature/sequencia' set up to track 'origin/feature/sequencia'.

hp@ALEXITA MINGW64 ~/helpdesk-data-center (feature/sequencia)
$ |
```

## 7. Rama *feature/mvc*

Se creó la rama *feature/mvc* a partir de *máster* mediante el comando `git checkout -b feature/mvc`, y se realizó el push al repositorio remoto con `git push -u origin feature/mvc`, el sistema confirmó la vinculación de la rama local con `origin/feature/mvc` y generó el enlace para la creación del pull request en GitHub, evidenciando que el trabajo quedó correctamente registrado en el repositorio.

```
Git CMD
C:\Users\hp>git branch
* master

C:\Users\hp>git checkout -b feature/mvc
Switched to a new branch 'feature/mvc'

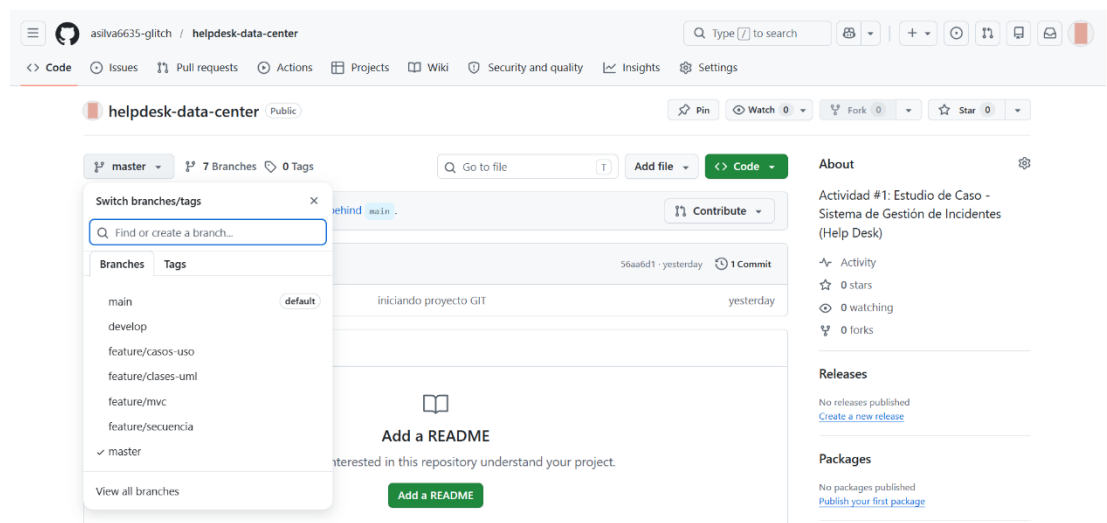
C:\Users\hp>git push -u origin feature/mvc
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/mvc' on GitHub by visiting:
remote:   https://github.com/asilva6635-glitch/helpdesk-data-center/pull/new/feature/mvc
remote:
To https://github.com/asilva6635-glitch/helpdesk-data-center.git
 * [new branch]      feature/mvc -> feature/mvc
branch 'feature/mvc' set up to track 'origin/feature/mvc'.

C:\Users\hp>git branch
* feature/mvc
  master

C:\Users\hp>git push -u origin feature/mvc
branch 'feature/mvc' set up to track 'origin/feature/mvc'.
Everything up-to-date

C:\Users\hp>
```

Después, el proyecto fue vinculado con un repositorio remoto en GitHub para almacenar los avances en la nube y mantener respaldo del trabajo realizado.





Finalmente, se utilizaron comandos como git add, git commit y git push para registrar y actualizar los cambios del proyecto de manera ordenada y segura.

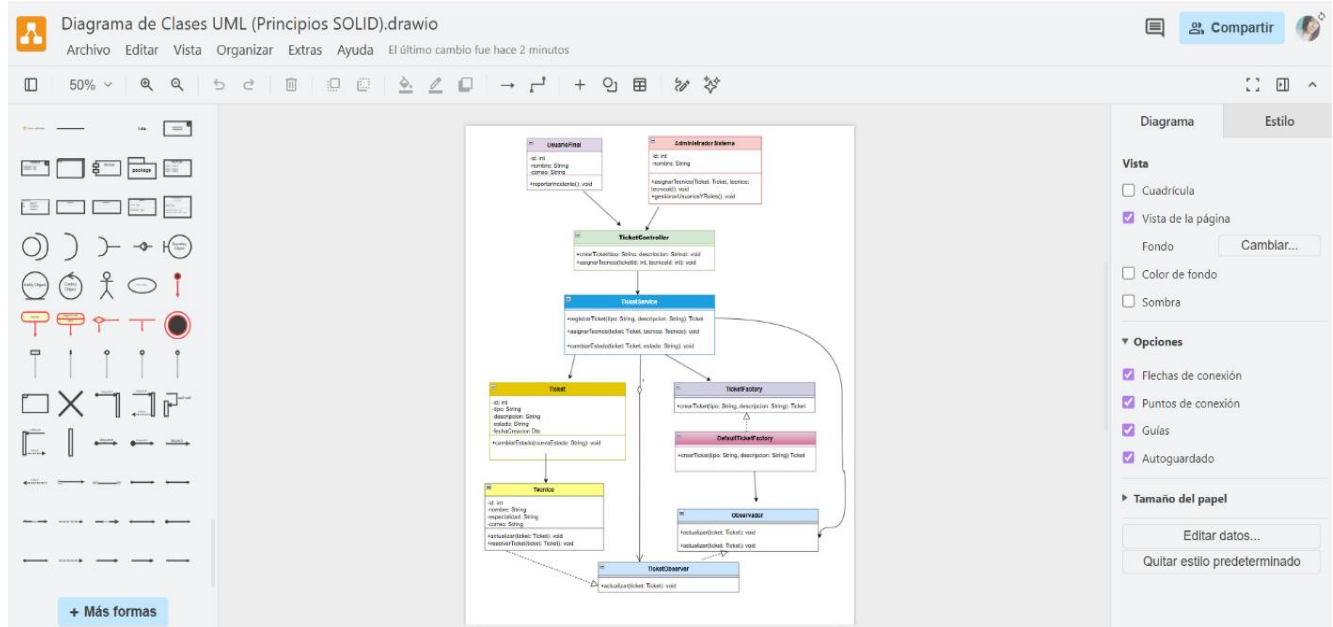
El repositorio público del proyecto se encuentra disponible en la siguiente dirección:

<https://github.com/asilva6635-glitch/helpdesk-data-center.git>

## 7. Proceso de modelado del diagrama de clases UML en draw.io.

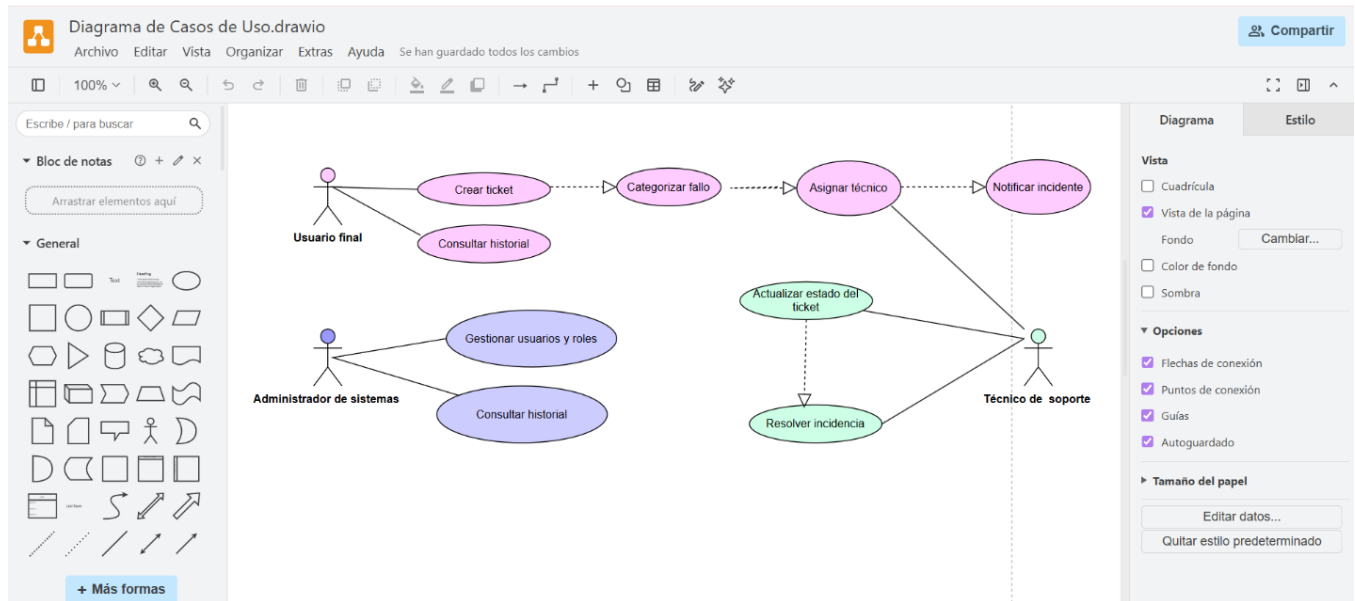
### 7.1. Diagrama de Casos de Uso.

Este diagrama muestra la interacción entre los actores principales (Usuario final, Técnico de soporte y Administrador de sistemas) y las funcionalidades clave del sistema Help Desk, como la creación, categorización y resolución de tickets.



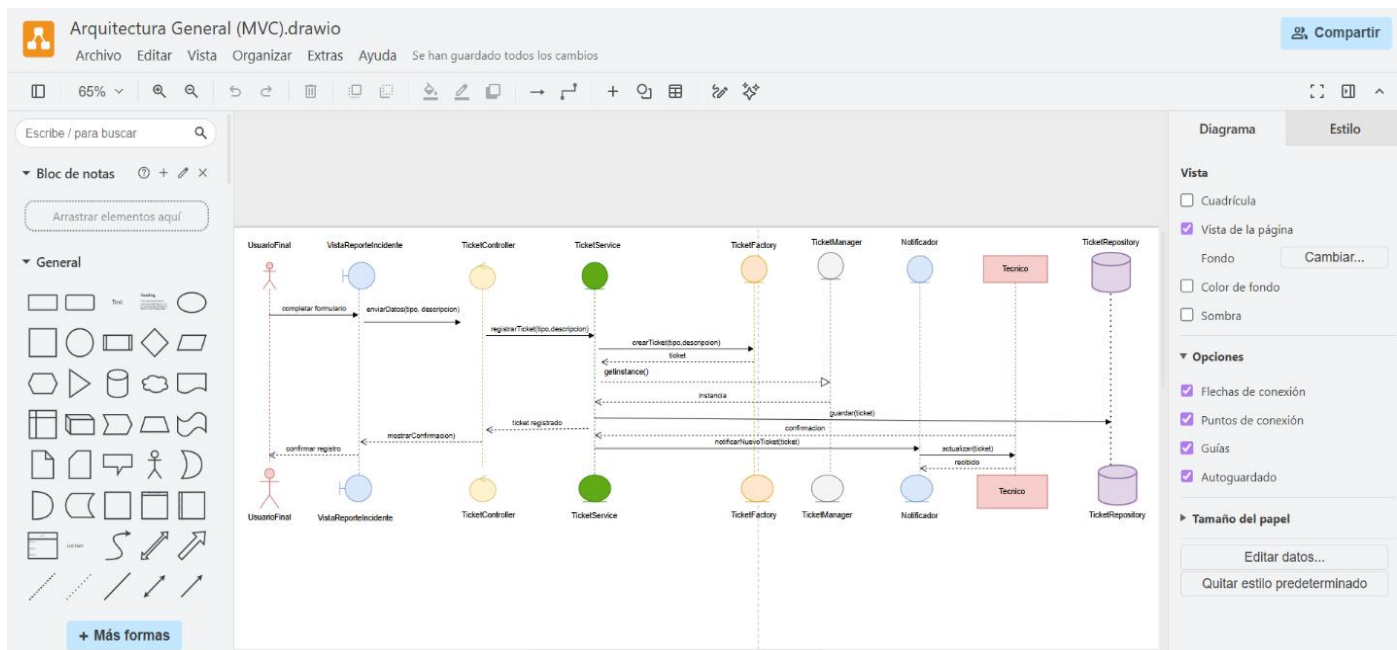
### 7.2. Diagrama de Clases UML.

Este diagrama representa la estructura del sistema Help Desk, mostrando las clases principales, sus atributos y métodos, así como las relaciones entre ellas. Se aplican los principios SOLID para garantizar mantenibilidad, escalabilidad y reutilización del código



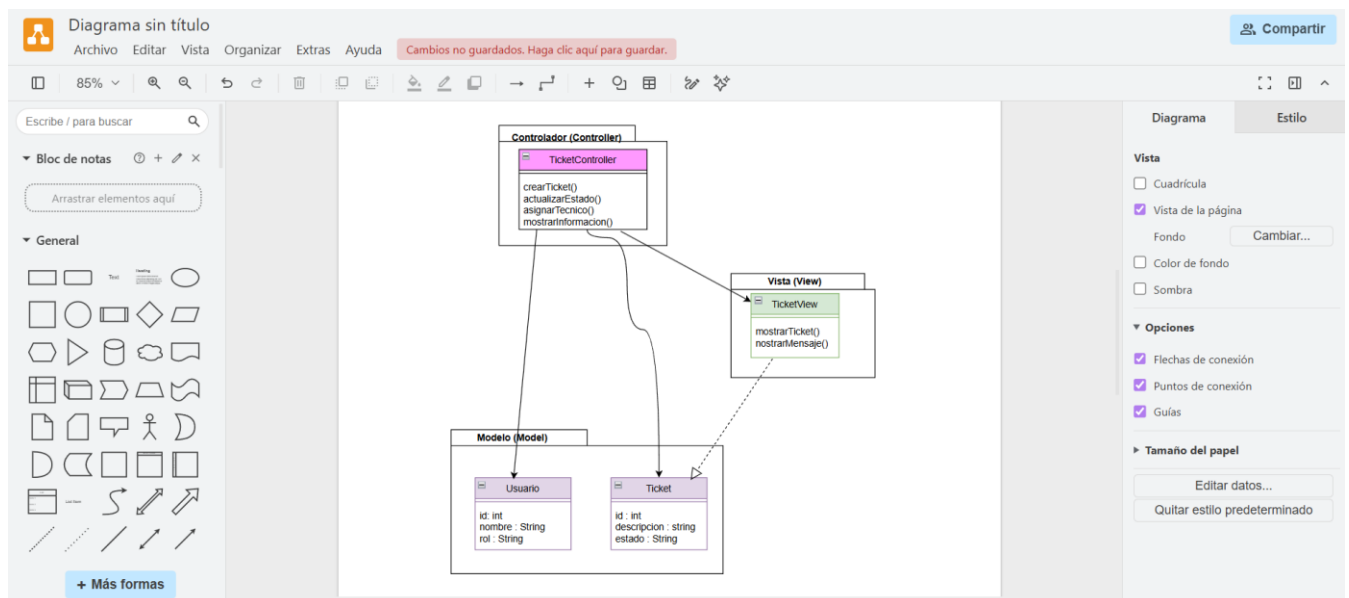
### 7.3. Diagrama de Secuencia.

Este diagrama muestra la interacción entre el Usuario Final, el Controlador, el Servicio de Tickets, el Gestor y el Notificador, evidenciando cómo se procesa un incidente desde su registro hasta la resolución por parte del técnico.



## 7.4.Arquitectura MVC

Este diagrama representa la organización del sistema Help Desk bajo el patrón Modelo-Vista-Controlador (MVC), separando la lógica de negocio, la interfaz gráfica y el control de procesos. Esta separación facilita la mantenibilidad, escalabilidad y el trabajo colaborativo entre desarrolladores.



## 8. Bibliografía

**Draw.io** – Herramienta para la elaboración de diagramas UML y arquitecturas.

<https://www.drawio.com/>

**Git** – Sistema de control de versiones distribuido.

<https://git-scm.com/>

**GitHub** – Plataforma para repositorios remotos y gestión colaborativa de proyectos.

<https://github.com/>